

Current Recursion and Tensor-Network Execution in PYAMPLICOL

Purpose. This note summarizes the matrix-element generation strategy used by AMPLICOL and the corresponding PYAMPLICOL implementation. The common idea is to avoid explicit Feynman-diagram enumeration. Instead, matrix elements are assembled from a directed acyclic graph (DAG) of off-shell currents. AMPLICOL emits Fortran code that numerically sweeps this current table. PYAMPLICOL builds the same logical DAG in Python, lowers bounded pieces to tensor-network expressions handled by Spenso, compiles the resulting parametric scalar/vector expressions with Symbolica evaluators, and can now execute the staged sweep from Rust through the PyO3 runtime `rusticol`.

1 The AmpliCol Current Recursion

For a fixed leading-color ordering, a current is labelled by the external subset it contains, the particle type carried by the off-shell line, and any spin/chirality information. Schematically, if I is a set or ordered interval of external colored legs, a current is computed as

$$\mathcal{J}_{t,\chi}^\alpha(I) = P^\alpha_\beta(t, p_I) \sum_{\substack{I=A \cup B \\ A \cap B = \emptyset}} \mathcal{V}^\beta_{\gamma\delta}(t_A, t_B \rightarrow t) \mathcal{J}_{t_A, \chi_A}^\gamma(A) \mathcal{J}_{t_B, \chi_B}^\delta(B). \quad (1)$$

Here t is the current type (gluon, quark, auxiliary tensor, etc.), χ is the Weyl chirality where relevant, $p_I = \sum_{i \in I} p_i$, P is the propagator, and $\mathcal{V}$ is the appropriate Feynman-rule tensor. The four-gluon vertex is treated through the auxiliary antisymmetric tensor route, so it is still represented by cubic current combinations.

AMPLICOL's optimized mode does not loop over full helicity configurations as independent computations. It weaves helicity information into the current table itself. Each external helicity state is a source current with a unique bit in an `ext_cur`-style bitset. An internal current is identified by

$$K(\mathcal{J}) = (t, \chi, \text{bin}(I), \text{ext_cur}), \quad (2)$$

where `bin(I)` records the external subset and `ext_cur` is the union of the source-current bits feeding the current. When a new vertex contribution produces the same key as an existing current, AMPLICOL appends that vertex to the existing current rather than creating a duplicate node. After all currents are constructed, `curr2amp` records the two endpoint currents whose contraction gives each retained helicity amplitude.

The generated Fortran library has the same layered structure:

```
subroutine evaluate_amp(p, amps)
  complex(kind=8) :: val_c(1:6, n_cur)
  complex(kind=8) :: int_c(1:6, n_vert)

  call compute_external_currents(pp, val_c)
  do size = 2, n_external_minus_one
    call compute_vertices_size(pp, val_c, int_c)
    call compute_currents_size(pp, val_c, int_c)
  end do
  call compute_amps(amps, val_c)
end subroutine
```

The arrays `val_c` and `int_c` are the numerical current and interaction tables. This is the performance-critical idea that PYAMPLICOL keeps: one topological sweep produces all retained amplitudes, and shared parent currents are evaluated once.

2 The pyAmpliCol DAG

PYAMPLICOL first builds an explicit Python representation of the same current table. For the leading-color milestone $q\bar{q} \rightarrow Z + ng$, the current identity is

$$K_{\text{PYAMPLICOL}} = (\text{pdg}, \text{labels}, \chi, \text{source_bits}). \quad (3)$$

The `source_bits` field is the direct analogue of AMPLICOL's `ext_cur`. Source currents are created for the allowed quark, antiquark, gluon, and massive-vector helicity states. Internal currents are then generated by increasing current size, with all interactions that feed the same key attached to one node.

```

for external helicity source s:
    add_current(key=(pdg_s, labels_s, chirality_s, bit_s), is_source=True)

for current_size in increasing_sizes:
    for compatible split A | B:
        result_key = combine_key(key[A], key[B], vertex_kind)
        result = add_or_reuse_current(result_key)
        add_interaction(left=A, right=B, result=result, vertex=vertex_kind)

for retained helicity ancestry:
    add_amplitude(left=current_with_Z_and_gluons, right=anti_current)

prune_dead_currents_after_helicity_filter()

```

At runtime, PYAMPLICOL mirrors the Fortran table layout with contiguous arrays:

$$C_{s,i,c} \equiv \text{current_rows}[s, i, c], \quad T_{s,v,c} \equiv \text{interaction_rows}[s, v, c],$$

where s is the phase-space sample in the current batch, i is a current id, v is an interaction id, and c is a component index. External wavefunctions fill source-current rows once per sample. Bounded evaluator blocks then fill interactions and derived currents stage by stage:

```

fill_source_currents(current_rows, momenta, helicity_sources)

for stage in current_size_layers:
    state = ParamBuilder.pack(current_rows, interaction_rows, momenta)
    interaction_rows[:, stage.vertices] = vertex_eval[stage](state)
    current_rows[:, stage.currents] = current_eval[stage](state)

amps = amplitude_eval(ParamBuilder.pack(current_rows, momenta))
me2 = normalization * sum(weight[h] * abs(amps[h])**2 for h in retained_h)

```

This is why the evaluator-only benchmark is meaningful: the expensive recursion is inside compiled evaluator calls, while Python only performs source wavefunction construction, parameter packing, and table transfers.

3 Spenso Tensor Networks and hep_lib

Spenso is used to express Lorentz, spinor, auxiliary-tensor, and eventually color contractions in a representation-aware way. A tensor-network expression contains named tensors with typed slots:

$$\mathcal{V}_{\mu\nu}{}^P(p_A, p_B) A^\mu B^\nu, \quad \text{or in code: } V(m("a"), m("b"), m("out")) * A(m("a")) * B(m("b")).$$

The expression is not expanded. It is a compact product of tensor factors with shared representation labels. The library `hep_lib` is the dictionary that tells Spenso what each tensor name means: dense model tensors, metric tensors, momentum-dependent propagators, and parametric rank-one tensors representing runtime inputs.

```

library = model.build_tensor_library() # returns TensorLibrary.hep_lib_atom()
builder = ParamBuilder()
mink = Representation.mink(4)

builder.register_rank1_tensor(
    library,
    tensor_name="current_A",
    representation=mink,
    head=("stage", "A"),
    length=4,
    role="block_current",
)
builder.register_rank1_tensor(
    library,
    tensor_name="p_A",
    representation=mink,
    head=("stage", "pA"),
    length=4,
    role="block_momentum",
    real_valued=True,
)

```

```

raw = V3g(mink("a"), mink("b"), mink("out"), p_A="p_A", p_B="p_B") \
    * TensorName("current_A")(mink("a")).to_expression() \
    * TensorName("current_B")(mink("b")).to_expression()

network = TensorNetwork(raw, library)
network.execute(library=library)
out_tensor = network.result_tensor(library)
evaluator = out_tensor.evaluator(params=builder.parameter_symbols())

```

The important point is that `hep_lib` stays fixed during the execution. New graph currents are not registered as permanent physics tensors. Instead, the current values are runtime parameters for a bounded block. Spensio executes the factorized tensor network, contracts the representation indices, and returns a parametric output tensor. Symbolica then compiles that parametric tensor to an evaluator. For high multiplicity, PYAMPLICOL interleaves this process: after a current or current-size layer is contracted, its numerical output is written back into the current table and becomes an input parameter for later blocks. This avoids one giant scalar expression and keeps the DAG close to AMPLICOL's execution model.

If explicit color tensors are introduced in future full-color modes, color simplification belongs before Spensio execution:

`raw ← idenso.simplify_color(raw),      then build TensorNetwork(raw, hep_lib).`

For the current leading-color milestone, color is already reduced to scalar factors associated with the ordered amplitude.

4 Example: $d\bar{d} \rightarrow Zgg$

The user-level process is written as

$$d(p_1) \bar{d}(p_2) \rightarrow Z(p_Z) g(p_3) g(p_4).$$

For the leading-color current recursion, the colored chain is treated in the canonical order

$$\bar{d} - d - g_3 - g_4,$$

with the color-singlet Z inserted on the quark line. The relevant source currents are

$$\mathcal{J}_{\bar{q}}(1, h_1), \quad \mathcal{J}_q(2, h_2), \quad \epsilon^\mu(3, h_3), \quad \epsilon^\nu(4, h_4), \quad \epsilon_Z^\rho(Z, h_Z).$$

The gluon subcurrents include

$$\mathcal{J}_g^\alpha(3, 4; h_3, h_4) = P_g^\alpha{}_\beta(p_3 + p_4) \mathcal{V}_{3g}^{\beta\mu\nu}(p_3, p_4) \epsilon_\mu(3, h_3) \epsilon_\nu(4, h_4), \quad (4)$$

$$\mathcal{J}_T^A(3, 4; h_3, h_4) = \mathcal{V}_{ggT}^{A\mu\nu} \epsilon_\mu(3, h_3) \epsilon_\nu(4, h_4), \quad (5)$$

where $\mathcal{J}_T$ is the auxiliary tensor current used to reproduce the four-gluon structure in larger examples. The quark line then forms

$$\mathcal{J}_q(2, 3) = P_q(p_2 + p_3) \mathcal{V}_{qqq}(\mathcal{J}_q(2), \epsilon(3)), \quad (6)$$

$$\mathcal{J}_q(2, 3, 4) = P_q(p_2 + p_3 + p_4) [\mathcal{V}_{qqq}(\mathcal{J}_q(2), \mathcal{J}_q(3, 4)) + \mathcal{V}_{qqq}(\mathcal{J}_q(2, 3), \epsilon(4))], \quad (7)$$

and the Z can be inserted before, between, or after the ordered gluons:

$$\mathcal{J}_q(2, Z) = P_q(p_2 + p_Z) \mathcal{V}_{qZq}(\mathcal{J}_q(2), \epsilon_Z), \quad (8)$$

$$\mathcal{J}_q(2, 3, Z) = P_q(p_2 + p_3 + p_Z) [\mathcal{V}_{qZq}(\mathcal{J}_q(2, 3), \epsilon_Z) + \mathcal{V}_{qqq}(\mathcal{J}_q(2, Z), \epsilon(3))], \quad (9)$$

with analogous terms leading to $\mathcal{J}_q(2, 3, 4, Z)$. The helicity amplitude is finally an endpoint contraction,

$$\mathcal{M}(h_1, h_2, h_3, h_4, h_Z) = \mathcal{J}_q(2, 3, 4, Z; h_2, h_3, h_4, h_Z) \cdot \mathcal{J}_{\bar{q}}(1, h_1). \quad (10)$$

Only retained helicity ancestries survive the filter. The reported squared matrix element has the same structure as in AMPLICOL:

$$|\mathcal{M}|_{\text{LC}}^2 = \mathcal{N}_{\text{AC}} \sum_{h \in H_{\text{kept}}} w_h |\mathcal{M}_h|^2, \quad (11)$$

where $\mathcal{N}_{\text{AC}}$ contains the leading-color normalization, couplings, symmetry factors, and initial-state averaging conventions matched to the Fortran implementation.

5 Rusticol: the Implemented Rust Runtime

The previous staged PYAMPLICOL runtime proved that the shared-current DAG could match Fortran AMPLICOL numerically, but its Python loop still moved current tables between many evaluator calls. The current implementation keeps Python as the process generator and validation driver, but moves the hot eager-DAG sweep into a PyO3 extension named `rusticol`. The generated artifact is now a self-contained process directory used by both runtimes:

```
rt_py = pyamplicol.load_process(process_dir, runtime="python")
values_py = rt_py.evaluate(momenta)

import rusticol
rt = rusticol.Runtime.load(process_dir)
values = rt.evaluate(momenta)
profile = rt.profile(momenta)
```

The Python and Rust runtimes read the same `process_manifest.json`, the same Symbolica evaluator manifests, and the same validation momenta. The Python path is kept as a transparent reference implementation. The Rust path is the performance path.

A process directory contains four classes of data. First, it records process and model metadata: external PDG order, momentum conventions, particle and vertex tables, couplings, masses, symmetry factors, and leading-color normalization. Second, it stores the shared-current DAG: source currents, current ids, dimensions, interaction stages, output slots, retained amplitude records, helicity weights, and color factors. Third, it stores the compiled Symbolica evaluator artifacts for each stage and for the final amplitude/raw-sum stage. Fourth, it contains standalone validation assets: `validation_momenta.json` with decimal-string momenta and a `check_standalone.py` script that imports only `rusticol`, plus `numpy` when available for the optimized f64 input path.

```
# Generated by pyAmpliCol.
process_dir/
  process_manifest.json
  manifest.json
  validation_momenta.json
  check_standalone.py
  compiled/           # C++/ASM shared libraries when present
  ... evaluator states ...
```

`rusticol.Runtime.load` validates the schema, checks that the current process family is supported, loads every Symbolica evaluator, computes direct state offsets for stage outputs, allocates reusable scratch buffers, and builds a fixed execution plan. The optimized f64 path accepts NumPy arrays with shape `(batch, n_external, 4)`. For each batch, Rust fills source wavefunctions and momentum sums, executes stage evaluators in topological order, writes the returned current components into the contiguous state table, executes the amplitude/raw-sum evaluator, applies the Fortran-matched normalization, and returns a NumPy array of matrix elements.

```
for point in batch {
    fill_source_currents(state_row, point);
    fill_momentum_sums(state_row, point);
}

for stage in stages {
    evaluated = stage.evaluator.evaluate_batch(state);
    assign_stage_outputs(state, evaluated, stage.output_offsets);
}

raw = amplitude_stage.evaluate_raw_sums(state);
me2 = normalization * raw;
```

The `profile` API reports the same split used in the benchmarks: source fill, momentum setup, stage evaluator time, output assignment, amplitude evaluator time, final reduction, total wall time, and memory snapshots where the platform exposes them.

The precision API follows Symbolica’s model. `evaluate` and `evaluate_with_prec(..., 16)` use the specialized native f64 path. `evaluate_with_prec(..., 32)` maps the serialized exact evaluator state to Symbolica’s `DoubleFloat` coefficients. Other positive precisions map to arbitrary-precision `Float` evaluators. The high-precision routes share the same stage orchestration code through generic Rust traits, while the f64 route remains specialized and avoids high-precision allocation overhead.

The artifact schema remains process-generic. The implemented fast family is leading-color $q\bar{q} \rightarrow Z + ng$; W^+ , e^+e^- , and ZZ probes currently raise explicit unsupported-process diagnostics.

6 Performance Results

Tables 1 and the markdown files `performance_summary.md` and `performance_summary_compiled_dag.md` report the current benchmark state for $d\bar{d} \rightarrow Z + ng$, $n = 1, \dots, 9$. All timing values are shown in monospace font. Green, orange, and red timing text follow the markdown convention: faster than Fortran AMPLiCOL, slower but below a factor two, and slower by a factor two or more. The blue rows are the Fortran AMPLiCOL references. The green highlighted rows are the present best production route, `rusticol` with C++ O3 stage evaluators.

Table 1: Combined performance summary for staged/Rusticol and compiled-DAG routes.

n	route	setup	gen [s]	wall [us/pt]	eval [us/pt]	notes
1	reference	AmpliCol	2.22	N/A	0.734	Fortran AmpliCol
1	staged/Rusticol	pyAmpliCol - JIT	0.016 (x0.01)	13.34(29) (x18.17)	0.607(17) (x0.83)	JIT; batch=16
1	staged/Rusticol	pyAmpliCol - C++	0.774 (x0.35)	13.01(12) (x17.72)	0.521(10) (x0.71)	C++; O3; chunk=None; batch=16
1	staged/Rusticol	pyAmpliCol - ASM	0.560 (x0.25)	14.03(30) (x19.12)	0.912(18) (x1.24)	ASM; O3; chunk=None; batch=16
1	staged/Rusticol	pyAmpliCol - Rusticol	0.984 (x0.44)	0.412(1) (x0.56)	0.1938(2) (x0.26)	Rusticol PyO3; C++ O3 process artifact; cached NumPy input; reusable stage scratch; single-chunk direct output; batch=1024
1	compiled-DAG	pyAmpliCol - compiled-DAG JIT	0.0483 (x0.02)	0.660 (x0.90)	0.108 (x0.15)	JIT; source and propagator-denominator aliases; batch 256; full-batch warm-up
1	compiled-DAG	pyAmpliCol - compiled-DAG C++	0.560 (x0.25)	0.768 (x1.05)	0.189 (x0.26)	C++; O3; chunk=None; batch 256; saved evaluator
1	compiled-DAG	pyAmpliCol - compiled-DAG ASM	0.297 (x0.13)	0.978 (x1.33)	0.403 (x0.55)	ASM; O3; chunk=None; batch 256; saved evaluator
2	reference	AmpliCol	2.20	N/A	1.64	Fortran AmpliCol
2	staged/Rusticol	pyAmpliCol - JIT	0.066 (x0.03)	16.15(15) (x9.85)	1.61(2) (x0.98)	JIT; batch=16
2	staged/Rusticol	pyAmpliCol - C++	1.86 (x0.85)	16.05(14) (x9.79)	1.16(2) (x0.70)	C++; O3; chunk=None; batch=16
2	staged/Rusticol	pyAmpliCol - ASM	0.940 (x0.43)	16.48(20) (x10.05)	2.16(2) (x1.32)	ASM; O3; chunk=None; batch=16
2	staged/Rusticol	pyAmpliCol - Rusticol	2.00 (x0.91)	0.984(1) (x0.60)	0.653(1) (x0.40)	Rusticol PyO3; C++ O3 process artifact; cached NumPy input; reusable stage scratch; single-chunk direct output; batch=1024
2	compiled-DAG	pyAmpliCol - compiled-DAG JIT	0.0941 (x0.04)	1.10 (x0.67)	0.421 (x0.26)	JIT; source and propagator-denominator aliases; batch 256; full-batch warm-up
2	compiled-DAG	pyAmpliCol - compiled-DAG C++	0.726 (x0.33)	1.22 (x0.74)	0.512 (x0.31)	C++; O3; chunk=None; batch 256; saved evaluator
2	compiled-DAG	pyAmpliCol - compiled-DAG ASM	0.379 (x0.17)	2.24 (x1.36)	1.51 (x0.92)	ASM; O3; chunk=None; batch 256; saved evaluator
3	reference	AmpliCol	2.31	N/A	4.55	Fortran AmpliCol
3	staged/Rusticol	pyAmpliCol - JIT	0.281 (x0.12)	22.67(42) (x4.98)	4.72(5) (x1.04)	JIT; batch=16
3	staged/Rusticol	pyAmpliCol - C++	5.69 (x2.46)	20.90(40) (x4.59)	3.06(4) (x0.67)	C++; O3; chunk=None; batch=16
3	staged/Rusticol	pyAmpliCol - ASM	1.52 (x0.66)	23.87(45) (x5.25)	6.17(8) (x1.36)	ASM; O3; chunk=None; batch=16
3	staged/Rusticol	pyAmpliCol - Rusticol	5.48 (x2.37)	2.783(13) (x0.61)	2.177(10) (x0.48)	Rusticol PyO3; C++ O3 process artifact; cached NumPy input; reusable stage scratch; batch=128
3	compiled-DAG	pyAmpliCol - compiled-DAG JIT	0.313 (x0.14)	3.18 (x0.70)	2.35 (x0.52)	JIT; source and propagator-denominator aliases; batch 256; full-batch warm-up
3	compiled-DAG	pyAmpliCol - compiled-DAG C++	2.84 (x1.23)	2.54 (x0.56)	1.70 (x0.37)	C++; O3; chunk=None; batch 256; saved evaluator
3	compiled-DAG	pyAmpliCol - compiled-DAG ASM	0.715 (x0.31)	5.51 (x1.21)	4.65 (x1.02)	ASM; O3; chunk=None; batch 256; saved evaluator
4	reference	AmpliCol	2.55	N/A	13.2	Fortran AmpliCol
4	staged/Rusticol	pyAmpliCol - JIT	0.252 (x0.10)	49.53(42) (x3.75)	28.37(17) (x2.15)	JIT; batch=16
4	staged/Rusticol	pyAmpliCol - C++	20.7 (x8.14)	27.78(13) (x2.10)	7.75(6) (x0.59)	C++; O3; chunk=None; batch=16
4	staged/Rusticol	pyAmpliCol - ASM	2.04 (x0.80)	36.30(54) (x2.75)	15.80(15) (x1.20)	ASM; O3; chunk=None; batch=16
4	staged/Rusticol	pyAmpliCol - Rusticol	19.9 (x7.81)	7.23(7) (x0.55)	6.10(6) (x0.46)	Rusticol PyO3; C++ O3 process artifact; cached NumPy input; reusable stage scratch; batch=128
4	compiled-DAG	pyAmpliCol - compiled-DAG JIT	0.301 (x0.12)	11.8 (x0.89)	10.9 (x0.82)	JIT; source and propagator-denominator aliases; batch 256; full-batch warm-up
4	compiled-DAG	pyAmpliCol - compiled-DAG C++	40.6 (x15.9)	16.8 (x1.27)	15.7 (x1.19)	C++; O3; chunk=None; batch 256; saved evaluator; monolithic C++ slower than JIT

Continued on next page

n route	setup	gen [s]	wall [us/pt]	eval [us/pt]	notes
4 compiled-DAG	pyAmpliCol - compiled-DAG ASM	1.02 (x0.40)	18.5 (x1.40)	17.4 (x1.32)	ASM; O3; chunk=None; batch 256; saved evaluator
5 reference	AmpliCol	3.22	N/A	34.3	Fortran AmpliCol
5 staged/Rusticol	pyAmpliCol - JIT	0.327 (x0.10)	373.6(2.8) (x10.89)	342.7(2.6) (x9.99)	JIT; batch=16
5 staged/Rusticol	pyAmpliCol - C++	54.3 (x16.87)	45.06(48) (x1.31)	20.83(26) (x0.61)	C++; O3; chunk=64; batch=128; chunk compile workers=10; two-stage save/load
5 staged/Rusticol	pyAmpliCol - C++ (O2)	50.7 (x15.76)	51.04(87) (x1.49)	23.82(42) (x0.69)	C++; O2; chunk=64; batch=16
5 staged/Rusticol	pyAmpliCol - ASM	10.2 (x3.16)	75.69(88) (x2.21)	48.74(72) (x1.42)	ASM; O3; chunk=64; batch=16
5 staged/Rusticol	pyAmpliCol - Rusticol	50.2 (x15.60)	19.63(28) (x0.57)	17.50(25) (x0.51)	Rusticol PyO3; C++ O3; chunk=64; cached NumPy input; reusable stage scratch; batch=128; two-stage save/load
5 compiled-DAG	pyAmpliCol - compiled-DAG JIT	0.435 (x0.14)	32.8 (x0.96)	31.0 (x0.90)	JIT; source and propagator-denominator aliases; batch 32; full-batch warm-up
5 compiled-DAG	pyAmpliCol - compiled-DAG C++	N/A	N/A	N/A	C++; O3; chunk=None; monolithic compile did not complete productively
5 compiled-DAG	pyAmpliCol - compiled-DAG ASM	1.94 (x0.60)	49.5 (x1.44)	47.6 (x1.39)	ASM; O3; chunk=None; batch 32; saved evaluator
6 reference	AmpliCol	4.55	N/A	84.1	Fortran AmpliCol
6 staged/Rusticol	pyAmpliCol - JIT	1.02 (x0.22)	1435(10) (x17.06)	1380(8) (x16.41)	JIT; batch=16
6 staged/Rusticol	pyAmpliCol - C++	151 (x33.23)	94.79(73) (x1.13)	56.36(49) (x0.67)	C++; O3; chunk=64; batch=128; chunk compile workers=10; two-stage save/load
6 staged/Rusticol	pyAmpliCol - C++ (O2)	140 (x30.74)	108.4(1.8) (x1.29)	68.5(1.6) (x0.81)	C++; O2; chunk=64; batch=16
6 staged/Rusticol	pyAmpliCol - ASM	21.4 (x4.70)	183.7(3.3) (x2.18)	141.2(2.7) (x1.68)	ASM; O3; chunk=64; batch=16
6 staged/Rusticol	pyAmpliCol - Rusticol	141 (x31.08)	55.63(26) (x0.66)	50.12(23) (x0.60)	Rusticol PyO3; C++ O3; chunk=64; cached NumPy input; reusable stage scratch; batch=128; two-stage save/load
6 compiled-DAG	pyAmpliCol - compiled-DAG JIT	1.26 (x0.28)	89.7 (x1.07)	87.7 (x1.04)	JIT; adaptive CPE unbounded; source and propagator-denominator aliases; batch 32; full-batch warm-up; max rel diff 5.1e-14 over 5 probe points
6 compiled-DAG	pyAmpliCol - compiled-DAG C++	814 (x179)	108 (x1.28)	106 (x1.26)	C++; O3; chunk=None; older completed probe; generation is not practical
6 compiled-DAG	pyAmpliCol - compiled-DAG ASM	3.99 (x0.88)	121 (x1.44)	120 (x1.42)	ASM; O3; chunk=None; batch 1024
7 reference	AmpliCol	9.38	N/A	207	Fortran AmpliCol
7 staged/Rusticol	pyAmpliCol - JIT	N/A	N/A	N/A	JIT backend bugged.
7 staged/Rusticol	pyAmpliCol - C++	417 (x44.44)	259.7(1.4) (x1.25)	194.7(1.2) (x0.94)	C++; O3; chunk=96; batch=128; chunk compile workers=10; two-stage save/load
7 staged/Rusticol	pyAmpliCol - C++ (O1)	219 (x23.35)	421.6(2.4) (x2.04)	348.9(1.9) (x1.69)	C++; O1; chunk=96; batch=16
7 staged/Rusticol	pyAmpliCol - ASM	36.2 (x3.86)	384.4(2.8) (x1.86)	320.2(2.4) (x1.55)	ASM; O3; chunk=96; batch=16
7 staged/Rusticol	pyAmpliCol - Rusticol	391 (x41.71)	197.8(4) (x0.96)	186.1(4) (x0.90)	Rusticol PyO3; C++ O3; chunk=96; cached NumPy input; reusable stage scratch; batch=128; two-stage save/load
7 compiled-DAG	pyAmpliCol - compiled-DAG JIT	3.90 (x0.42)	268 (x1.29)	265 (x1.28)	JIT; adaptive CPE unbounded; source and propagator-denominator aliases; batch 24; full-batch warm-up; max rel diff 3.7e-13 over 3 probe points
7 compiled-DAG	pyAmpliCol - compiled-DAG C++	N/A	N/A	N/A	C++; O3; chunk=None; not attempted after monolithic n=5 C++ compile failed productively
7 compiled-DAG	pyAmpliCol - compiled-DAG ASM	11.1 (x1.19)	369 (x1.78)	364 (x1.76)	ASM; O3; chunk=None; batch 512
8 reference	AmpliCol	25.0	N/A	567	Fortran AmpliCol
8 staged/Rusticol	pyAmpliCol - JIT	N/A	N/A	N/A	JIT backend bugged.
8 staged/Rusticol	pyAmpliCol - C++	1.06e3 (x42.24)	642.6(2.1) (x1.13)	491.8(1.4) (x0.87)	C++; O3; chunk=96; batch=128; chunk compile workers=10; two-stage save/load
8 staged/Rusticol	pyAmpliCol - C++ (O1)	577 (x23.09)	997.2(3.3) (x1.76)	885.3(2.9) (x1.56)	C++; O1; chunk=96; batch=16
8 staged/Rusticol	pyAmpliCol - ASM	85.6 (x3.42)	1117.0(4.6) (x1.97)	1004.9(4.3) (x1.77)	ASM; O3; chunk=96; batch=16
8 staged/Rusticol	pyAmpliCol - Rusticol	1.02e3 (x40.90)	487.7(9) (x0.86)	464.4(6) (x0.82)	Rusticol PyO3; C++ O3; chunk=96; cached NumPy input; reusable stage scratch; batch=128; two-stage save/load
8 compiled-DAG	pyAmpliCol - compiled-DAG JIT	16.6 (x0.67)	3601 (x6.35)	3596 (x6.34)	JIT; source and propagator-denominator aliases; batch 512

Continued on next page

n route	setup	gen [s]	wall [us/pt]	eval [us/pt]	notes
8 compiled-DAG	pyAmpliCol - compiled-DAG C++	N/A	N/A	N/A	C++; O3; chunk=None; not attempted after monolithic n=5 C++ compile failed productively
8 compiled-DAG	pyAmpliCol - compiled-DAG ASM	34.7 (x1.39)	1432 (x2.53)	1430 (x2.52)	ASM O3; inline external wavefunctions; max rel diff 1.4e-13 over 3 probe points
9 reference	AmpliCol	48.0	N/A	2520	Fortran AmpliCol
9 staged/Rusticol	pyAmpliCol - JIT	N/A	N/A	N/A	JIT backend bugged.
9 staged/Rusticol	pyAmpliCol - C++	2.83e3 (x58.83)	1504.9(11.7) (x0.60)	1252.7(9.6) (x0.50)	C++; O3; chunk=96; batch=128; chunk compile workers=10; two-stage save/load
9 staged/Rusticol	pyAmpliCol - ASM	349 (x7.27)	2688.2(5.5) (x1.07)	2459.1(4.5) (x0.98)	ASM; O3; chunk=64; batch=16
9 staged/Rusticol	pyAmpliCol - Rusticol	2.83e3 (x58.83)	1164.8(9) (x0.46)	1120.9(1.0) (x0.44)	Rusticol PyO3; C++ O3; chunk=96; cached NumPy input; reusable stage scratch; batch=128; repackaged from existing C++ O3 artifact in 103 s; two-stage save/load
9 compiled-DAG	pyAmpliCol - compiled-DAG JIT	122 (x2.54)	8056 (x3.20)	8051 (x3.19)	JIT; inline external wavefunctions; fresh Fortran probe currently fails with reference-side SIGSEGV
9 compiled-DAG	pyAmpliCol - compiled-DAG C++	N/A	N/A	N/A	C++; O3; chunk=None; not attempted after monolithic n=5 C++ compile failed productively
9 compiled-DAG	pyAmpliCol - compiled-DAG ASM	191 (x3.97)	10705 (x4.25)	10700 (x4.25)	ASM; O3; chunk=None; batch 512; peak RSS 8.01 GiB

7 Why This Matches AmpliCol but Remains Python-Friendly

The Python layer owns graph construction, process metadata, deterministic phase-space steering, artifact generation, evaluator caching, and validation against AMPLICOL. It does not perform the hot current recursion component by component in production. The two production execution routes are now:

Python staged route: source fill $\rightarrow$ bounded stage evaluators $\rightarrow$ amplitude/raw-sum evaluator,
Rusticol route: one PyO3 call $\rightarrow$ Rust source fill and staged sweep $\rightarrow$ amplitude/raw-sum evaluator.

Thus PYAMPLICOL keeps AMPLICOL’s essential optimization - shared currents across helicity parents and a topological current-table sweep - while replacing generated Fortran subroutines by saved Spensio/Symbolica evaluator artifacts. The `rusticol` path removes the remaining Python orchestration cost by loading the same process directory and executing the sweep in Rust.

The fully monolithic compiled-DAG route is still valuable as a research backend: it inlines the current graph into a single multi-output evaluator and reaches excellent low-multiplicity performance. At larger multiplicity, however, its code shape no longer resembles the staged current-table schedule closely enough. The best current production compromise is therefore Python generation plus a Rust eager-DAG runtime over saved Symbolica stage evaluators.